第八节 加密解密、集成

肖逸飞
xyf1989@uestc.edu.cn

课程安排

1. 软件工程与软件建模
2. 敏捷开发与项目协作
3. 开发环境规范与测试
4. AI辅助开发
5. 需求设计与程序框架
6. 打包解包及其关键技术
7. 压缩解压及其关键技术
8. 加密解密与系统集成
9. 软件发布与项目收尾
10. 项目答辩与成果展示

实验难度分级和评分标准

基本要求

各小组“独立”实现一款数据备份软件（对应**基础分总分40分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分总分**）。

文件类型支持（10分）：支持特定文件系统的特殊文件（管道/软链接/硬链接等）

元数据支持（10分）：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份（各5分）：允许用户筛选需要备份的文件（路径/类型/名字/时间/尺寸）

打包解包（10分）：将所有备份文件拼接为一个大文件保存

压缩解压（10分）：通过文件压缩节省备份文件的存储空间

加密解密（10分）：由用户指定密码，将所有备份文件均加密保存

图形界面（10分）：实现友好易用的GUI界面

定时备份（10分）：允许用户进行设置，进行周期性定时备份和数据淘汰

实时备份（15分）：自动感知用户文件变化，进行自动备份

网络备份（30分）：将数据备份软件从单机模式扩展为网盘模式（10分），还涉及到的功能包括：用户管理（5分）、元数据管理（5分）、传输加密（5分）、增量备份（5分）等。

其它功能：视功能难度讨论加分。

实验难度分级和评分标准

基本要求

各小组“独立”实现一款数据备份软件（对应**基础分总分40分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分总分**）。

文件类型支持（10分）：支持特定文件系统的特殊文件（管道/软链接/硬链接等）

元数据支持（10分）：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份（各5分）：允许用户筛选需要备份的文件（路径/类型/名字/时间/尺寸）

打包解包（10分）：将所有备份文件拼接为一个大文件保存

压缩解压（10分）：通过文件压缩节省备份文件的存储空间

加密解密（10分）：由用户指定密码，将所有备份文件均加密保存

图形界面（10分）：实现友好易用的GUI界面

定时备份（10分）：允许用户进行设置，进行周期性定时备份和数据淘汰

实时备份（15分）：自动感知用户文件变化，进行自动备份

网络备份（30分）：将数据备份软件从单机模式扩展为网盘模式（10分），还涉及到的功能包括：用户管理（5分）、元数据管理（5分）、传输加密（5分）、增量备份（5分）等。

其它功能：视功能难度讨论加分。

加密&解密

对称加密和散列算法

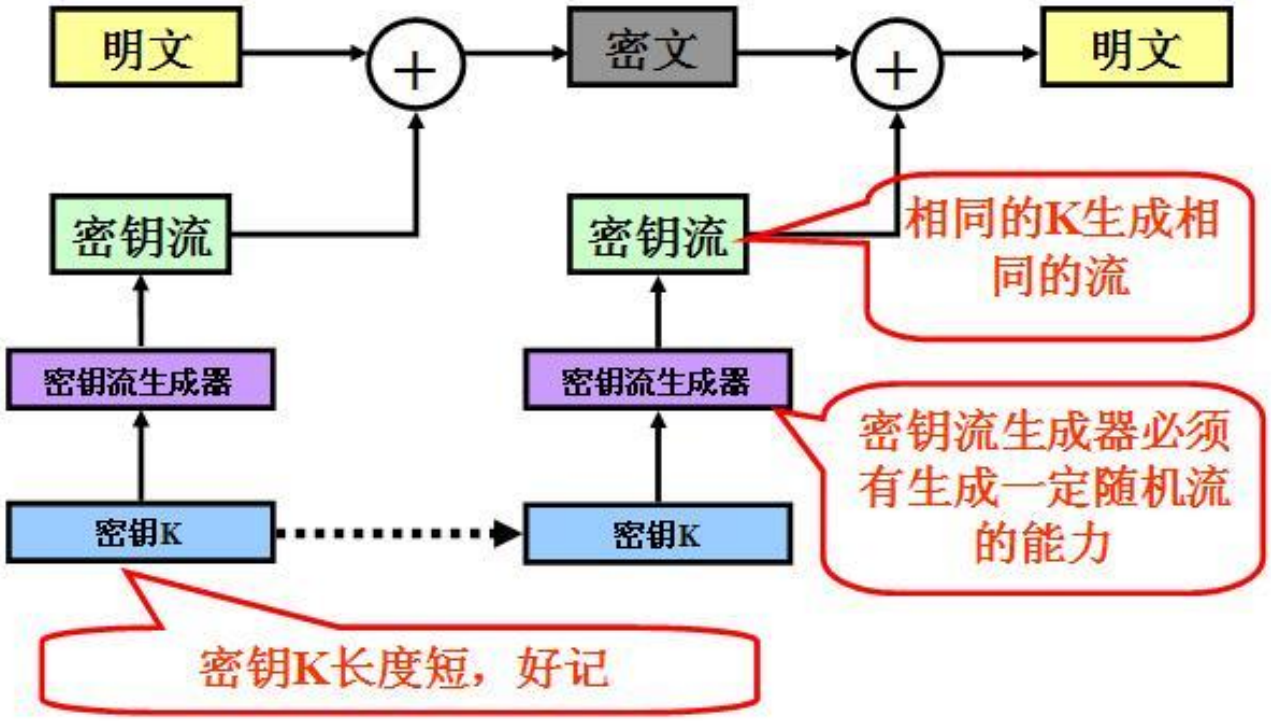
对于数据备份而言，主要考虑**对称加密**和**散列算法**。

散列算法也称为摘要算法，如MD5、SHA1等。

简单对称加密方式可考虑流密码。

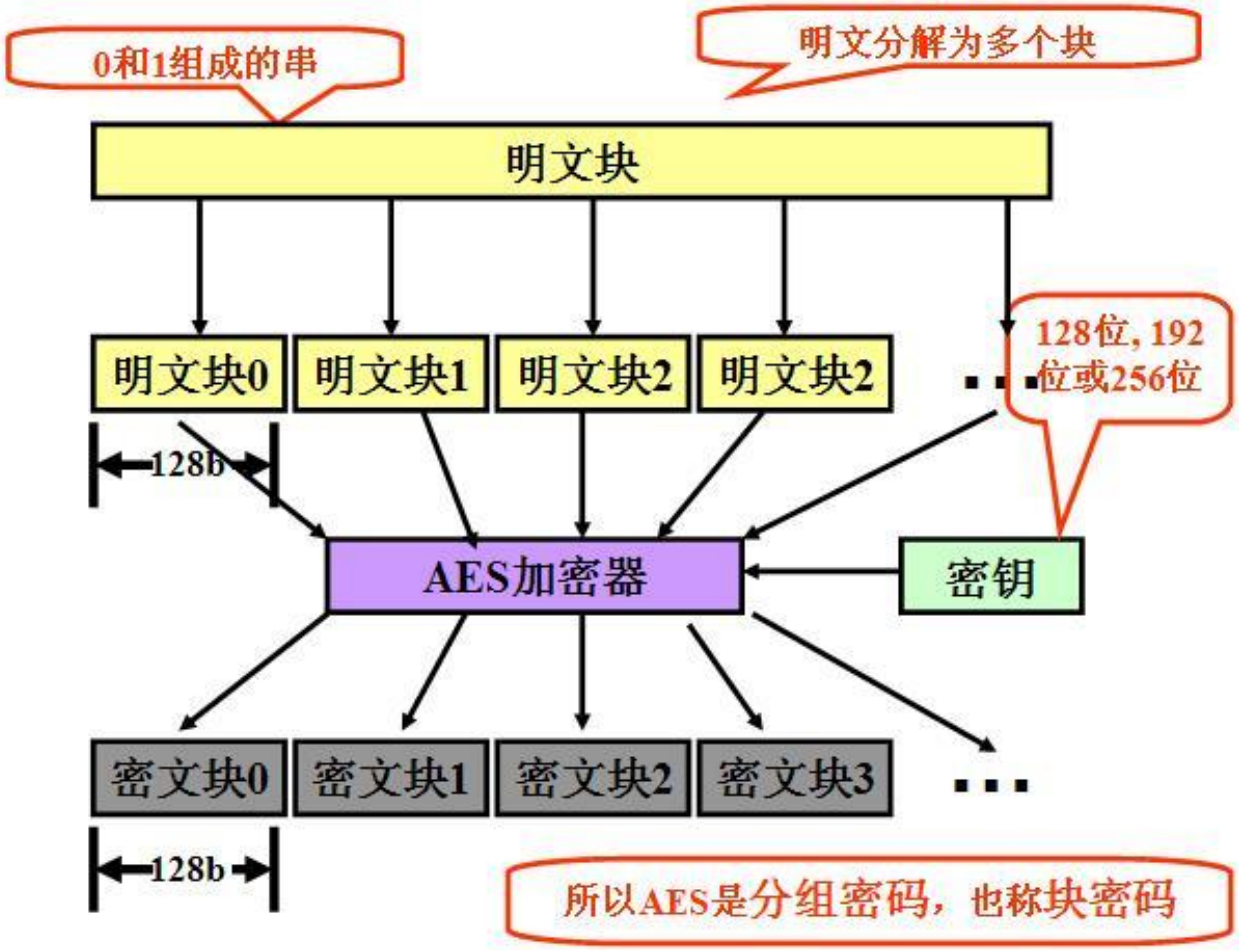
更优的对称加密方式为分组加密，如DES、AES等。

流密码

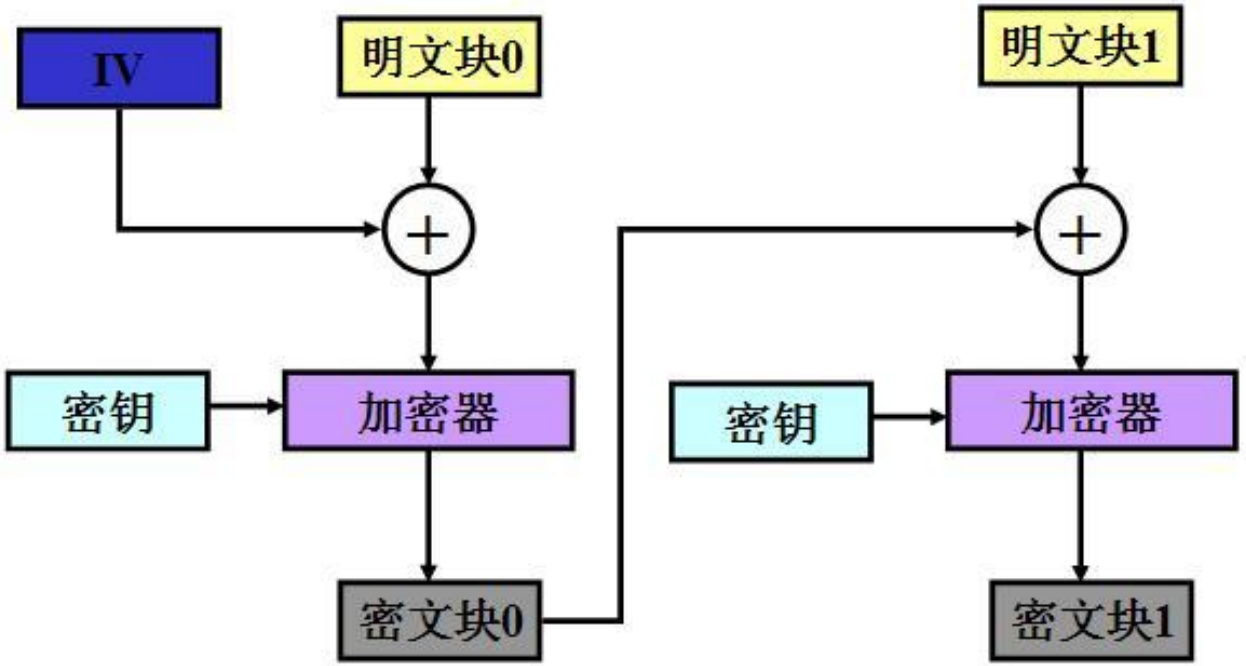


常见流密码算法：RC4，ChaCha20以及AES的流密码模式等

AES加密


0和1组成的串
明文分解为多个块
明文块
明文块0
明文块1
明文块2
明文块2
128位, 192位或256位
128b
AES加密器
密钥
密文块0
密文块1
密文块2
密文块3
128b
所以AES是分组密码，也称块密码

AES加密-CBC模式



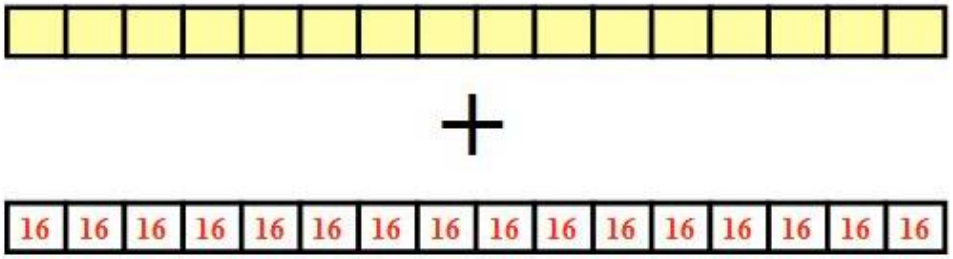
分组加密中的块补齐问题

一个AES块为16个字节（128位），需要考虑数据补齐问题。
主要分两种情况：

1、若数据的最后一块不够16个字节，则将最后一个数据块补齐为16个字节，填补的每个字符的内容为**填补字节的个数**。



2、若数据的最后一块刚好16个字节，则新增一块，该块中的每个字符的内容均为**块大小**，即16。



加解密常用库——OpenSSL

OpenSSL中的MD5、AES相关API：

MD5_Init
MD5_Update
MD5_Final
AES_set_encrypt_key
AES_set_decrypt_key
AES_ecb_encrypt
AES_cbc_encrypt
AES_cfb128_encrypt
AES_ofb128_encrypt

加解密常用库——OpenSSL

OpenSSL中的MD5、AES相关API：

MD5_Init
MD5_Update
MD5_Final
AES_set_encrypt_key
AES_set_decrypt_key
AES_ecb_encrypt
AES_cbc_encrypt
AES_cfb128_encrypt
AES_ofb128_encrypt

这些哈希和加密API在新版的OpenSSL手册中已经不再推荐使用。

推荐使用OpenSSL最新的EVP API。EVP API对哈希和加密算法进行了更高层的封装，屏蔽了算法之间使用上的细微差别。

加解密常用库——OpenSSL

OpenSSL EVP API中的常用函数：

EVP_MD_CTX_new
EVP_MD_CTX_free
EVP_MD_fetch
EVP_MD_free
EVP_DigestInit_ex
EVP_DigestUpdate
EVP_DigestFinal_ex

EVP_CIPHER_CTX_new
EVP_CIPHER_CTX_free
EVP_CIPHER_fetch
EVP_CIPHER_free
EVP_EncryptInit_ex
EVP_EncryptUpdate
EVP_EncryptFinal_ex
EVP_DecryptInit_ex
EVP_DecryptUpdate
EVP_DecryptFinal_ex

加解密常用库——OpenSSL

OpenSSL EVP API中的常用函数：

EVP_MD_CTX_new
EVP_MD_CTX_free
EVP_MD_fetch
EVP_MD_free
EVP_DigestInit_ex
EVP_DigestUpdate
EVP_DigestFinal_ex

EVP_CIPHER_CTX_new
EVP_CIPHER_CTX_free
EVP_CIPHER_fetch
EVP_CIPHER_free
EVP_EncryptInit_ex
EVP_EncryptUpdate
EVP_EncryptFinal_ex
EVP_DecryptInit_ex
EVP_DecryptUpdate
EVP_DecryptFinal_ex

官方手册：
https://docs.openssl.org/master/man3/

参考加密解密代码
库的使用

封装成库

封装成库

为什么要封装成库?

使程序的核心逻辑实现与界面实现分离开来
从而实现各自的技术栈和工作方向的独立

可以使用C++来暴露库接口吗?

可以，但并不推荐。

什么是ABI

ABI（Application Binary Interface）：**应用二进制接口**，是指不同程序或库之间的对接规范，以确保二进制可兼容性。ABI规范通常包含：

调用约定（函数调用及传参方式）

```
int __cdecl add_numbers(int a, int b, int c);

// 调用时的栈布局（从右向左压参）
push c      // 第三个参数
push b      // 第二个参数
push a      // 第一个参数
call add_numbers
add esp, 12 // 调用者清理栈
```

数据类型（格式和大小）

```
// 32位 vs 64位系统的差异
typedef struct {
    int32_t value1;    // 4字节
    void* pointer;     // 4字节(32位) vs 8字节(64位)
    int16_t value2;    // 2字节
} ExampleStruct;

// 内存布局差异：
// 32位: [value1(4)][pointer(4)][value2(2)][padding(2)] = 12字节
// 64位: [value1(4)][padding(4)][pointer(8)][value2(2)][padding(6)] = 24字节
```

寄存器使用规则
堆/栈结构
名字装饰约定
错误和异常处理
模块初始化和终止过程

什么是ABI

不同的编程语言和平台有各自的ABI：

- Linux有IA32和x86-64 ABI
- C/C++ 在不同平台上有不同的ABI标准
- Java 和 .NET 均有各自的平台无关的ABI标准

C没有平台无关的ABI标准，各常用平台均有自己的C ABI标准，如：Linux、Windows、macOS、POSIX、ARM等。现行C++标准没有覆盖ABI，而是由各平台自行制定。

符合同一ABI的程序或库之间，可以在二进制级别互相调用和链接。
否则，则称之为二进制不兼容。

使用C暴露C++接口

在使用C++进行库开发时，对外暴露C接口通常更合适：

- C接口兼容性更好，很多语言都提供了调用C接口的机制。
- C接口没有C++的名称修饰问题（mangle）。
- C接口的ABI标准更为简单、通用。
- C接口不存在C++的异常传播问题。
- C接口受众更广。

例如：C++支持函数重载，因此编译器需要根据函数名和参数类型来生成唯一的符号名，这个过程称为名称修饰（或名字改编）。例如，函数int foo(int)和double foo(double)在C++中会被修饰成不同的符号（如_Z3fooi和_Z3food）。

```
// C++库接口 - 问题多多
class Database{
public:
    Database(const::&);// 名称修饰
    ~Database();// 名称修饰 + 异常安全问题
    std::vector<Record> query(const std::string& sql);// 可能抛出异常
    // ABI问题：std::vector和std::string的布局依赖编译器版本
    };// 客户端必须用相同编译器、相同C++标准库版本
```

参考接口和库的实现

界面实现

参考界面实现和库的调用

实验环节

完成数据备份软件的:

加密解密部分

完成各模块的系统集成

（推荐）完成单元测试